SmartHouse

Software Engineering Capstone | FivePlusOne Devs | CSC 312

Design Model

Personalised architectural consultation, house concepts and interactive visual exploration.

CURRENT BASELINE: FRONTEND PROTOTYPE

React + Vite interface; browser-local data; rule-based room layout; SVG floor plans and exterior elevations; Three.js interactive house model. No login, API server or database is required.

This document explains the prototype as implemented on 11 October 2026. It links user needs to interface behaviour, component responsibilities, data structures, generation rules and validation evidence.

Design intent

Understand how a household lives, translate its confirmed brief into a coherent concept, and present the same house through plan, exterior, 3D model and cost estimate. The design experience takes priority over backend infrastructure in this prototype.

Document map

02 Actors, use cases and consultation workflow

03 Frontend architecture and component responsibilities

04 Data structures and persistence model

05 Interaction sequence and state transitions

06 Layout generation and worked house example

07 Interface model and interactive 3D design

08 Validation, limitations and future evolution

01 / User and interaction model

A guided architectural brief is the starting point for design, rather than a catalogue search.

Actor	Prototype responsibility
Homeowner / demo user	Create a project; answer questions; confirm a brief; explore, revise and download a concept.
Demo manager	Edit illustrative library designs and rates locally. This is a demo interface, not a secured administrator role.
Architect / cost practitioner	Potential reviewers of spatial rules and estimate assumptions; no separate professional workflow is implemented.

Main use-case path

1. Start a project

Name a home project and create a local draft.

↓

2. Complete consultation

Land; budget and household; daily life; style and priorities.

↓

3. Review and confirm

Validate required answers and acknowledge concept assumptions.

↓

4. Explore your house

Rotate the 3D house, inspect exterior views or open the floor plan.

↓

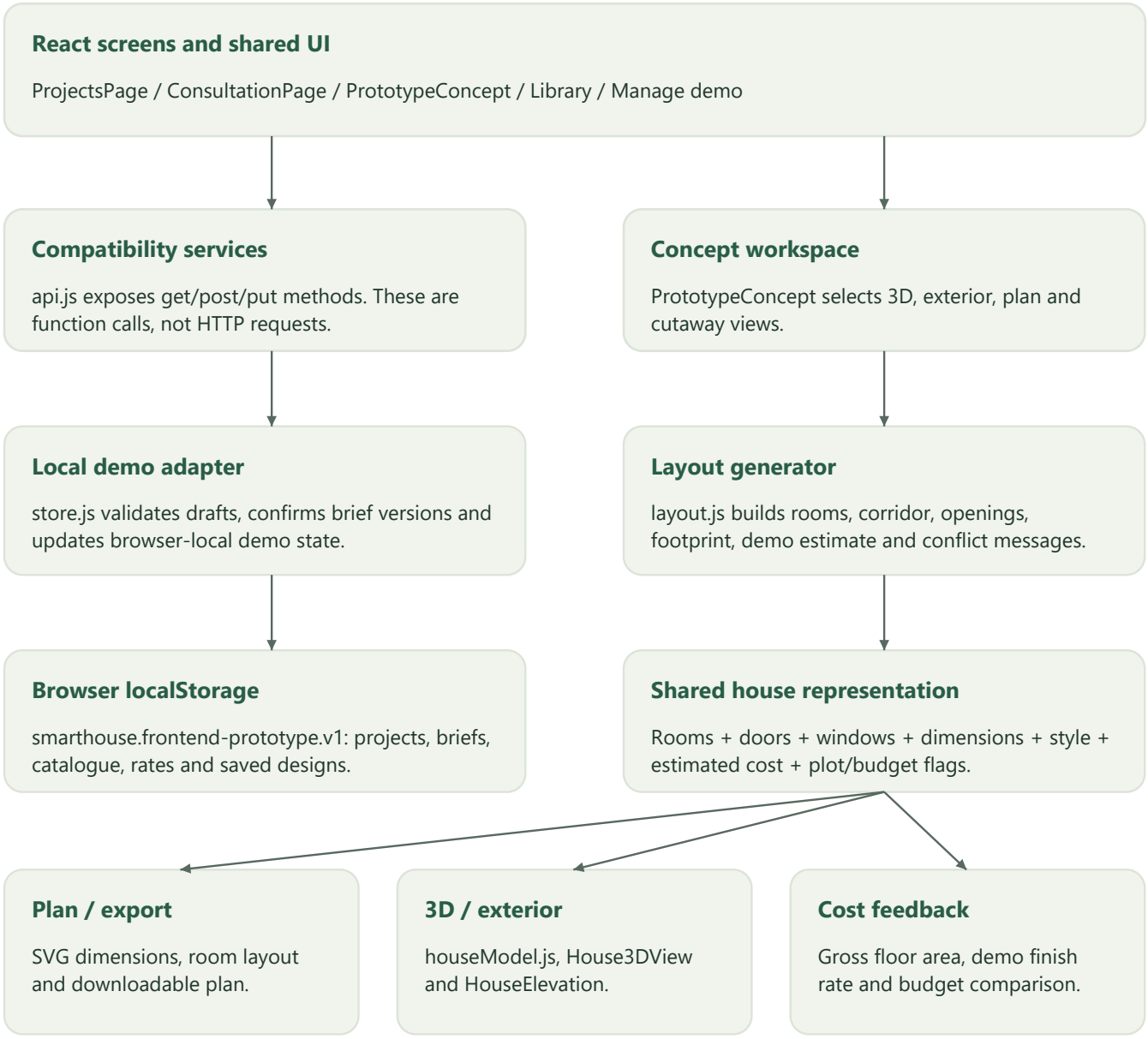
5. Revise or download

Edit the brief and reconfirm, or export SVG plan / JSON concept data.

Conditional questions ask about an office when working from home, guest needs when guests stay often, and specific accessibility requirements. Unsupported details are recorded for review rather than silently treated as generated features.

02 / Frontend architecture

Current implementation boundary: one browser application with local persistence and generated geometry.

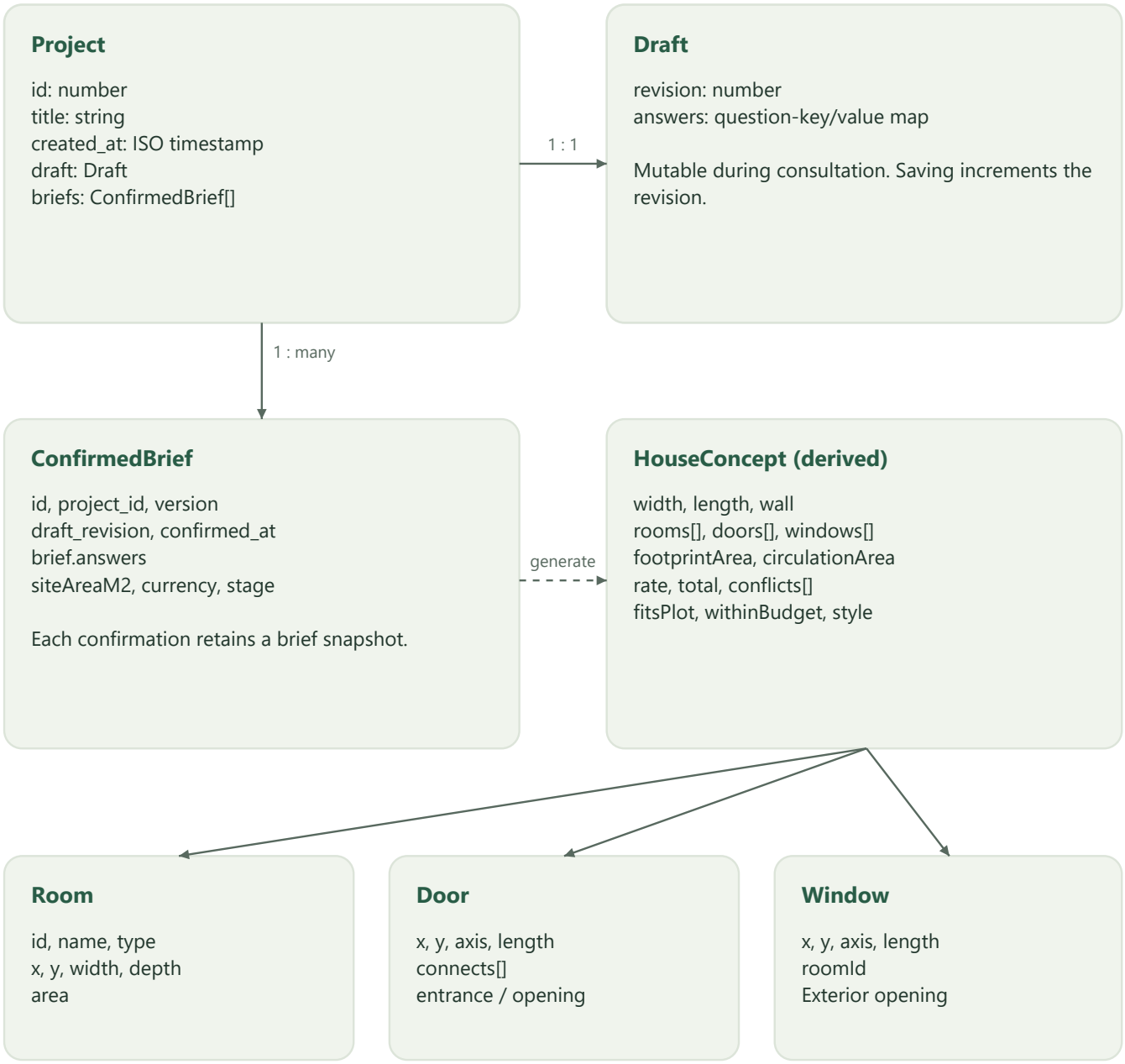


Geometry is derived from a confirmed brief when the concept view renders. The browser stores the brief, not a versioned mesh file. All visual views consume the generated house object; Three.js creates meshes from those dimensions.

Boundary: the legacy Express/PostgreSQL implementation and target backend documents remain in the repository, but are not part of the active frontend run command or data path.

03 / Data and class model

Browser-local project state and generated objects have different lifecycles.

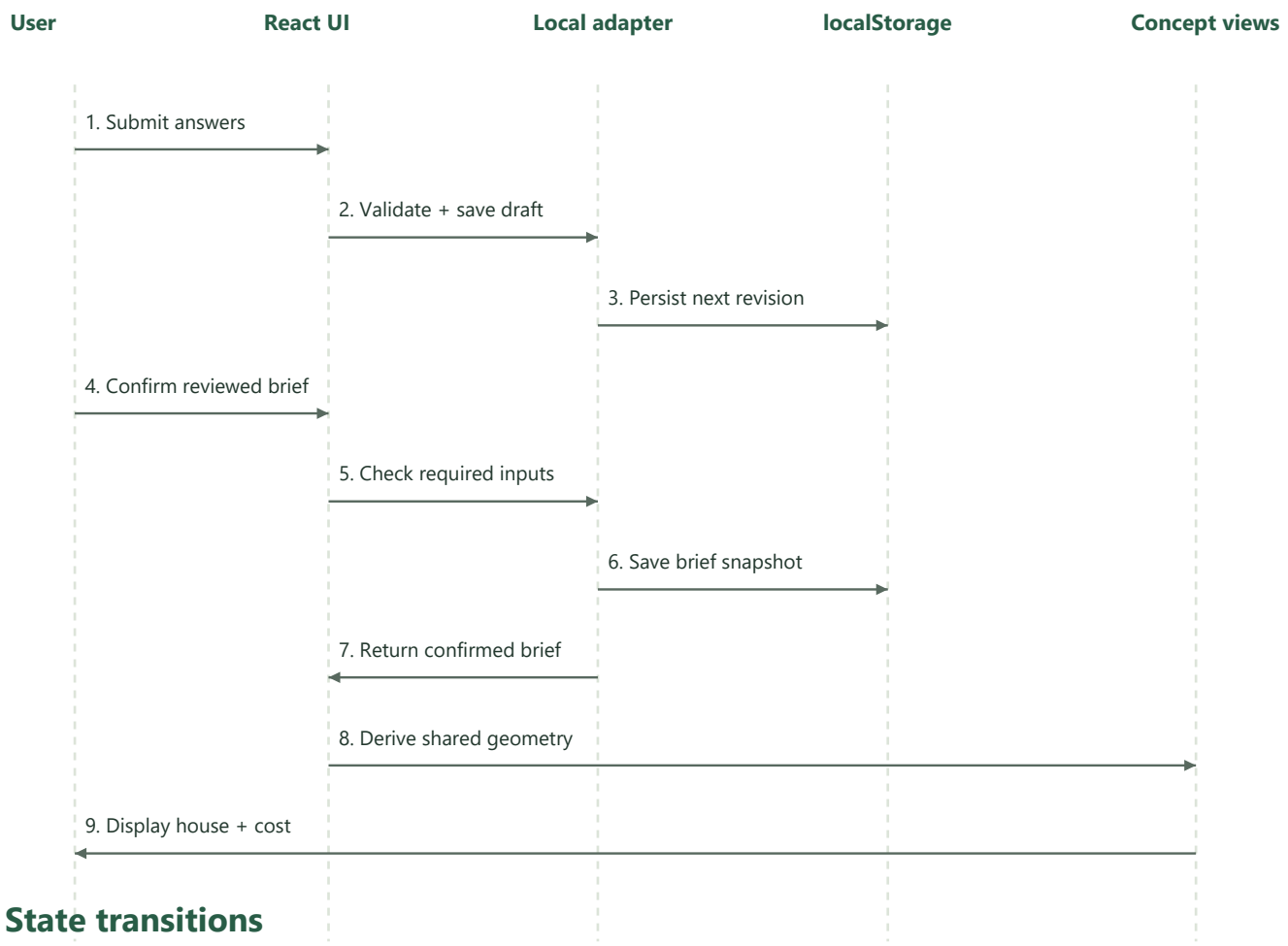


The local store also contains plans[], saved plan IDs, demo finish rates and the last catalogue search. No user authentication or cross-device ownership model is active.

Saved brief versions are retained, but layout geometry is recomputed from the current generator. Rates used by custom concepts are currently fixed demo values in layout.js; library rate edits apply to catalogue estimates. A production snapshot model would need to pin geometry and pricing explicitly.

04 / Interaction sequence

Saving a draft and confirming a brief are local operations with explicit validation and revision checks.



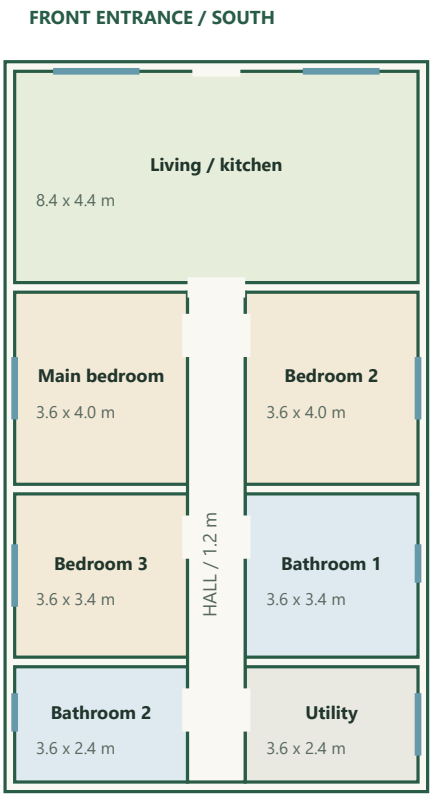
State transitions

State / event	Behaviour
Draft -> saved draft	Answers persist locally; revision increments; stale expected revisions are rejected.
Saved draft -> confirmed brief	Required and conditional answers must be complete. Rectangular plots are supported; stated sloped terrain blocks confirmation.
Confirmed brief -> design workspace	The concept is generated locally. Plot and budget conflicts remain visible; they do not silently remove requested rooms.
Edit brief -> reconfirm	A new brief snapshot is created. The previous confirmed brief is retained; users can return to the current design view.

05 / House-generation model

A constrained layout pattern gives the prototype coherent circulation and a matching model.

The generator creates a front living zone and a 1.2 m central hallway. Bedrooms, bathrooms and an optional dedicated office are placed in paired bays. An unmatched bay becomes laundry/storage. A separate kitchen receives a direct opening to the living area.



Plan schematic from generated coordinates

Worked example	Value
Plot / rooms	20 x 30 m; 3 bedrooms; 2 bathrooms
Style / finish	Traditional; basic finish
Footprint	8.8 x 15.2 m
Gross floor area	133.76 square metres
Demo estimate	R1,270,720 at R9,500 / m2
Budget	R1,500,000
Result	Fits demo envelope; below demo budget

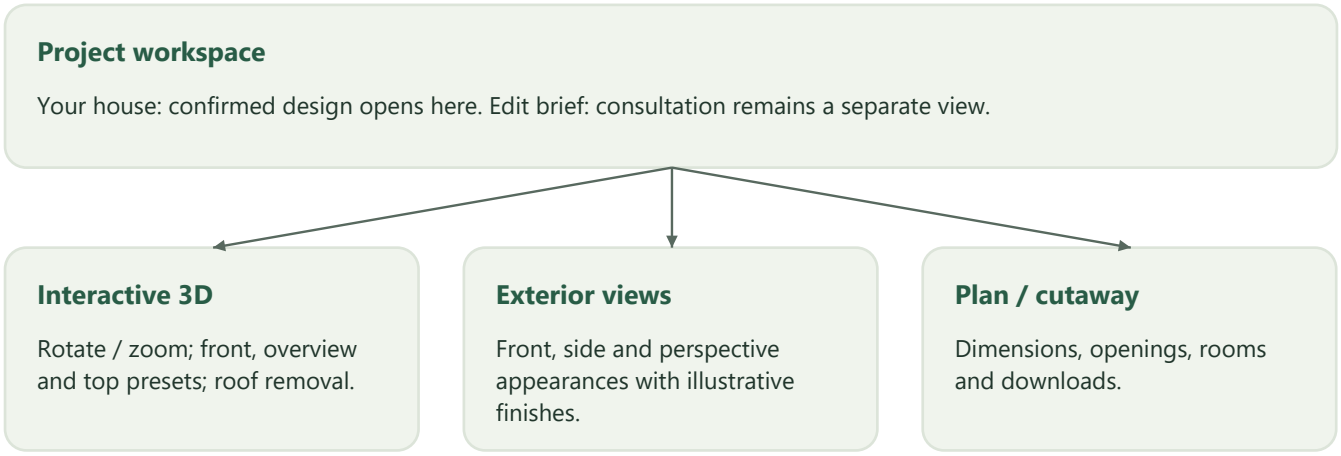
Generation and cost rules

1. Expand supported room requirements into fixed-size room components.
2. Pair private rooms around the hallway and add door/window coordinates.
3. Compute the gross footprint, including circulation and wall allowances.
4. Estimate cost = rounded gross floor area x illustrative finish rate.
5. Flag an oversized footprint or an estimate above the budget.

Demo envelope: 1 m allowance per plot edge; this is not a verified planning rule. East/west road access swaps frontage and depth for the fit comparison. Stated terrain may be unknown; no terrain analysis is performed. Furniture and clearance drawings are indicative.

06 / Interface and visual model

The project workspace separates editing from exploration to reduce repeated scrolling.



3D model responsibilities

Component	Design responsibility
houseModel.js	Create floor slabs, exterior wall meshes, window/door openings, partitions, furniture and style-dependent roof geometry.
House3DView.jsx	Set up camera, lighting and OrbitControls; resize to the viewport; expose rotate/zoom controls; lower walls when the roof is removed.
Lifecycle / fallback	Dispose geometries, materials, controls and renderer resources when the view unmounts. If WebGL fails, offer a 2D exterior view and retry.

UI decisions that support the workflow

Compact project header and model viewport; separate Your house / Edit brief navigation; collapsed review details; sticky consultation actions; loading/retry feedback; visible stale-concept message after edits; keyboard-operable camera buttons alongside mouse and touch gestures.

The 3D viewer is loaded as a separate module. Models are built in the browser from local geometry, not downloaded from an asset server. External photorealistic rendering, free-form room editing and a construction-ready CAD model are not implemented.

07 / Verification and evolution

Prototype evidence, known limits and a staged path toward the full architectural platform.

Verification area	Implemented checks
Local workflow	Create, save, confirm and reopen without a network; stale revisions; incomplete briefs; storage failure feedback.
Layout geometry	Supported bedroom/bathroom combinations; finite dimensions; footprint bounds; no room overlap; hallway access; exterior windows.
3D consistency	Room-floor count matches the plan; windows are real mesh openings; roof height/style checks; interior survives roof removal.
Build / code quality	Frontend build and ESLint checks; prototype test runner contains eight test cases. Browser interaction and professional design review remain separate evaluation tasks.

Current limitations

The active generator uses one adaptable layout pattern, fixed component sizes and illustrative area rates. It does not search arbitrary layouts, interpret the imported floor-plan image dataset, enforce all stated lifestyle priorities, verify accessibility clearances, or perform structural/regulatory checks. Local storage is device-specific and has no authentication or cross-device sync.

Future design increments

Next prototype work: additional layout patterns, more flexible room sizing, clearer material options, stronger circulation checks and user feedback.
Later platform work: persisted geometry/pricing snapshots, validated configuration releases, quantity-based costing, protected projects, professional review and downloadable concept packages. Backend implementation is deferred until the prototype workflow is agreed.

Repository basis

Product baseline: docs/SmartHouse_Revised_Project_Proposal.md.
Current status: docs/PROTOTYPE_PROGRESS.md.
Implementation: client/src/demo/store.js, layout.js, houseModel.js; client/src/pages/ProjectsPage.jsx and PrototypeConcept.jsx; client/src/components/plans/House3DView.jsx and HouseElevation.jsx.
Evidence definitions: client/tests/prototype.test.cjs.
Future specifications: docs/REQUIREMENTS.md, ARCHITECTURE.md and ERD.md.

Interpretation note: future specifications contain authentication and backend requirements that are intentionally outside the current frontend-only baseline. This document describes design models, not approved house construction drawings.